ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/376675658

Adaptive Swarm Intelligent Offloading Based on Digital Twin-assisted Prediction in VEC

Article *in* IEEE Transactions on Mobile Computing · January 2023

DOI: 10.1109/TMC.2023.3344645

CITATIONS
37

READS
104

7 authors, including:



Liang Zhao
Shenyang Aerospace University
313 PUBLICATIONS **5,338** CITATIONS
SEE PROFILE



Tianyu Li
Northeastern University
4 PUBLICATIONS **40** CITATIONS
SEE PROFILE



Enchao Zhang
University of Electro-Communications
9 PUBLICATIONS **258** CITATIONS
SEE PROFILE



Shaohua Wan
University of Electronic Science and Technology of China
346 PUBLICATIONS **14,725** CITATIONS
SEE PROFILE

All content following this page was uploaded by Enchao Zhang on 07 April 2025.

The user has requested enhancement of the downloaded file.

Adaptive Swarm Intelligent Offloading Based on Digital Twin-assisted Prediction in VEC

Liang Zhao, *Member, IEEE*, Tianyu Li, Enchao Zhang, Yun Lin, *Senior Member, IEEE*, Shaohua Wan, *Senior Member, IEEE*, Ammar Hawbani, and Mohsen Guizani, *Fellow, IEEE*

***Abstract*—Vehicular Edge Computing (VEC) is the transportation version of Mobile Edge Computing (MEC). In VEC, task offloading enables vehicles to offload computing tasks to nearby Roadside Units (RSUs), thereby reducing the computation cost. Recent trends in task offloading cause a proliferation of studies in academia. However, the existing offloading schemes still face many challenges, such as high-dynamic network topology, massive and complex data, dynamic scenes with high-speed vehicles and low-latency requirements. Digital Twin (DT)-based VEC is emerging as a promising solution. It monitors the state of the VEC network in real time through mappings and interactions between the physical and virtual entities. Consequently, the task offloading scheme can make more reasonable offloading decisions at the physical layer and further improve the efficiency of VEC. Above all, we propose a VEC computing offloading scheme, namely, Adaptive Swarm Intelligent Offloading Scheme Based on Digital-Twin-Assisted PRediction In VEC (STRIVE). The VEC network architecture is established to combines DT with an improved Generative Adversarial Network (GAN). The powerful prediction ability of GAN is used to assist in constructing DT in the pre-processing phase, reducing the size of the decision space. To adapt to the dynamic nature of VEC, we establish an adaptive model to adjust the real-time parameter under various scenarios. Then, we deploy an improveD genetIc simulatEd annealing-baSEd particLe swarm optimization (DIESEL) algorithm to task offloading decision-making, which can provide reliable computing services for vehicles at a lower cost. The simulation results demonstrate that the proposed scheme can effectively reduce computing delay and energy consumption compared with its counterparts.**

***Index Terms*—Digital twin, generative adversarial network, mobile edge computing, swarm intelligence, task offloading, vehicular edge computing.**

Manuscript received 10 April 2023; revised 5 December 2023; accepted 11 December 2023. Date of publication 19 December 2023; date of current version 2 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62372310, in part by Liaoning Province Applied Basic Research Program under Grant 2023JH2/101300194, and in part by LiaoNing Revitalization Talents Program. Recommended for acceptance by O. Yagan. *(Corresponding authors: Ammar Hawbani; Yun Lin.)*

Liang Zhao, Tianyu Li, Enchao Zhang, and Ammar Hawbani are with the School of Computer Science, Shenyang Aerospace University, Shenyang 110136, China (e-mail: lzhao@sau.edu.cn; 13998798119@163.com; enchaozhang1998@163.com; anmande@ustc.edu.cn).

Yun Lin is with the College of Information and Communication Engineering, Harbin Engineering University, Harbin 150009, China (e-mail: linyun@hrbeu.edu.cn).

Shaohua Wan is with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518110, China (e-mail: shaohua.wan@ieee.org).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, UAE (e-mail: mguizani@ieee.org).

Digital Object Identifier 10.1109/TMC.2023.3344645

I. Introduction

MOBILE Edge Computing (MEC) is a promising paradigm that migrates remote cloud computing resources to the edge of networks [1]. MEC enables tasks to be offloaded to nearby edge servers, reducing the computation pressure on mobile devices. Nowadays, an explosion of mobile applications such as healthcare, Virtual Reality (VR), Augmented Reality (AR), and autonomous driving require MEC as their supporting platform [2]. MEC can reduce service latency and network congestion by providing computing resources to end users [3].

Vehicular Edge Computing (VEC) is the traffic version of MEC in the road scenario, and task offloading is a key service [4]. In VEC task offloading, vehicles can offload computation tasks to the nearby Roadside Units (RSU) or surrounding vehicles with computation capabilities, thus reducing execution delay and energy consumption. Compared with traditional offloading of mobile devices, VEC faces a significant challenge due to the high-speed mobility of vehicles in road scenarios, coupled with scene variability. The traditional offloading scheme is Vehicle to Infrastructure (V2I) offloading [5], where the tasks are offloaded from the vehicle to the RSU. However, the V2I offloading framework cannot take advantage of surrounding vehicle resources. In urban scenarios, the RSUs are overloaded due to the high traffic density. Nowadays, vehicle-to-vehicle (V2V) offloading attracts extensive attention from academics, which has more advantages over traditional V2I [6]. However, there is a critical challenge in the current research on the V2V offloading scheme. Due to the complexity of the V2V offloading model, it must cope with the time-varying dynamic network topology, as V2V communication may be disrupted due to variations in the speed of the vehicle.

One possible method for fixing the above problem is to construct a pre-processing space by considering the dynamics of vehicles and the diversity of scenarios. As a concept beyond reality, the Digital Twin (DT) can build virtual representations of physical entities in cyberspace. This state mapping between physical and virtual dimensions gives users an in-depth understanding of the system under investigation and fundamentally alters the design and engineering process [7]. DT can monitor the whole VEC network in real-time to cope with time-varying network topologies [8]. By combining DT technology with existing artificial intelligence in VEC, DT serves as a pre-processing space for predicting vehicle trajectories, addressing the problem of high-speed mobility for vehicles and reducing the decision

1536-1233 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

space [9]. Therefore, we establish a virtual simulation platform using DT to solve the problem of the lack of a simulation platform.

Due to the heterogeneity of devices and diversity of traffic scenarios in the VEC network, traditional machine learning and mathematical methods for task offloading heavily rely on expertise, resulting in time-consuming processes [10]. An adaptive solution is crucial for various dynamic scenarios. Researchers primarily turn to Reinforcement Learning (RL) for task offloading due to its rapid problem-solving abilities in large decision spaces and multi-dimensional state spaces. However, RL is primarily trained for static scenarios, which has obvious advantages in fields such as chess. Because it has fixed rules and developers can set reward mechanisms through the simple win-lose conditions of the game [11]. In VEC, models trained by RL have limitations in task offloading decision-making as they cannot adapt to changes. The offloading scheme needs retraining when the coverage of RSU, the number of vehicles, or the load degree of vehicles is changed, making it impractical. When deploying some typical heuristic algorithms in VEC scenarios, challenges arise for the high-speed mobility of vehicles and the requirements of real-time algorithms [12].

By comprehensively considering the dynamics of VEC and the model complexity of various environments, we propose a novel task offloading scheme, namely, Adaptive Swarm Intelligent Offloading Scheme Based on Digital-Twin-Assisted PRediction In VEC (STRIVE). This scheme effectively reduces the complexity of the V2V offloading model and the maximizes computation resource utilization for densely populated urban scenarios. Applying DT and Wasserstein Generative Adversarial Networks (WGAN) to VEC establishes a DT model based on the future mapping of raw data. WGAN uses Wasserstein distance to replace Jensen-Shannon (JS) divergence in GAN, which can accurately quantify the difference between two distributions [13]. Vehicle trajectory data can be predicted to calculate the relative distance between vehicles by using WGAN, thereby reducing the decision space and decision diversity and further reducing the complexity of V2V offloading [14]. We also apply an improveD genetIc simulatEd annealing-baSEd particLe swarm optimization (DIESEL) algorithm to the task offloading decision-making in adaptive scenarios. The proposed methodology aims to reduce the overall average cost of the system. The contributions of this paper are summarized as follows:

- To cope with the dynamic nature of the VEC network, we established a comprehensive distributed task offloading architecture based on DT-assisted VEC. An adaptive model is proposed to meet the various environmental circumstances according to the dynamic multi-scale features and diversity in the system.
- The combination of DT and WGAN is applied to calculate the future relative distance of vehicles to maximize the utility of DT and reduce the system complexity. The DT model based on the future mapping of raw data is established. This reduces the V2V offloading decision space, further enhancing the system efficiency and stability. The overall process can be defined as an integral part of pre-processing.
- An adaptive task offloading scheme for VEC, namely, DIESEL is devised based on the DT model established by future trajectory data of vehicles. DIESEL combines the advantages of the powerful local search capability of Particle Swarm Optimization (PSO) and the stochastic global search capability of Genetic Algorithm (GA). In addition, it also combines with DT-based adaptive annealing coefficients in Simulated Annealing (SA) algorithm to cope with dynamic scenarios. The proposed scheme can provide reliable real-time services for vehicles with minor system cost.

The remainder of this paper is structured as follows: The related work is presented in Section II. Section III introduces the system model and problem formulation. The specifics of the STRIVE task offloading scheme are described in Section IV. Discussions and experimental analyses are provided in Section V. In Section VI, the conclusion is presented.

II. Related Work

In this section, we review the literature that utilizes various techniques for V2I and V2V offloading tasks. In addition, several investigations utilizing the DT technology for task offloading are presented. We also highlight the innovative aspects of our research.

A. Task Offloading in V2I

V2I offloading is a common approach in VEC systems. Many existing studies employ Deep Reinforcement Learning (DRL) methods for V2I offloading. For instance, Zhao et al. [15] employ an enhanced Actor-Critic (AC) algorithm to solve the offloading problem about dependent tasks generated by vehicles in V2I context. Li et al. [16] introduce a task offloading algorithm based on DRL that no longer requires complete a priority knowledge of communication states or computational resources, thus optimizing the allocation of tasks and resources in VEC systems. Some research also takes into account the issue of resource allocation. Li et al. [17] propose a Task Partitioning and Scheduling Algorithm (TPSA) to determine the load distribution on RSUs, and develop an DRL-based computational method for determining task offloading strategies. Ouyang et al. [18] optimize the offloading problem into a vehicle scheduling sub-problem and a computational resource optimization sub-problem, proposing a semi-online offloading algorithm based on Dueling-DQN.

Besides DRL methods, various heuristic algorithms and game theory approaches have also been applied in V2I offloading. Xia et al. [19] propose a Multi-Cell and Multi-Time Slots Offloading (MCMTSO) algorithm to minimize the total delay of offloading strategies with sub-tasks from vehicles. Ko et al. [20] propose a Brief-based Task Offloading Algorithm (BOTA) to ensure tasks are offloaded to the most suitable edge cloud and sub-channel. Zhang et al. [21] module the competition and cooperation among requesting vehicles, edge servers, and cloud services as a Stackelberg game and propose a search algorithm based on GA to find the optimal offloading method.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

B. Task Offloading in V2V

In recent years, V2V offloading has become a primary method in VEC. Unlike traditional V2I offloading, vehicles with computing capabilities can act as edge servers to address the issues of high computational load and limited coverage of RSUs. Machine learning and deep learning methods are widely utilized in V2V offloading. For instance, Guo et al. [22] design a V2V task offloading algorithm based on dual Long Short-Term Memory (LSTM) to address the significant fluctuations in dynamic network performance. Dai et al. [23] develop a learning-based V2V computation offloading algorithm that enhanced the capabilities and trust awareness of service vehicles. Fan et al. [24] deploy deep learning method to alleviate network congestion and improve traffic load balancing. This approach aims to reduce the complexity of Software-Defined Network (SDN) control and enhance offloading efficiency in VEC network. DRL with its characteristic of continuous exploration in vast decision spaces, is similarly suitable for the aforementioned problems. Shi et al. [25] develop a RL-based computation offloading solution for blockchain smart contracts to ensure the reliability of resource allocation in task offloading. Shi et al.[26] implement a novel V2V partial offloading solution using DRL method to address the issues of idle computational resource conservation and high mobility of vehicles.

However, in highly dynamic scenarios with large decision spaces, DRL training struggles to converge. Some literature has resorted to traditional heuristic algorithms to address these issues. Liu et al. [27] investigate the task offloading issue in V2V networks supporting MEC under unstable connections and propose a distributed online instability-aware computing offloading heuristic algorithm to enhance the quality of service. Xiao et al. [28] use the Generalized Benders Decomposition (GBD)-based scheme to solve the problem of energy scarcity and information security issues in the context of VEC task offloading. Feng et al. [29] consider the issue of channel resource allocation caused by differences in vehicle and RSU channels in C-V2X. By decomposing the overarching problem of offloading decisions and resource allocation into three sub-problems, a solution was sought using a genetic algorithm based on PC5 port.

C. Task Offloading by DT Technology

DT, as an emerging technology, possesses the capability to map physical entities into the digital network of DT. It can also be regarded as one or more significant and interdependent digital mapping systems, which are widely employed in a variety of industries such as engineering, manufacturing, medical analysis, and construction [30]. In recent years, there has been an increasing amount of literature studying the combination of DT and task offloading. Liu et al. [31] propose a DT-assisted scheme that includes target selection and intelligent task offloading, aiming to address the issue of mobile users offloading to collaborative edge servers.

Currently, there are numerous publications that apply DT technology to VEC offloading problems. Xu et al. [32] by updating the DT of vehicles, a multi-user offloading system is designed using DRL algorithm to enhance the quality of service for users. Zhang et al. [33] utilize DT to predict vehicle historical data and propose a collaborative task offloading algorithm based on DRL, which reduces the task offloading decision space. Cao et al. [34] construct a five-objective optimization model for vehicle user offloading issues and propose an enhanced CMA-ES algorithm based on guided points in DT environment. Zhao et al. [35] address the issue of excessively large decision spaces by combining an improved clustering algorithm with DT and utilizing a DRL algorithm for offloading decisions in VEC network. Yuan et al. [36] utilize DT for real-time data collection and digital representation of the physical operating environment in vehicular networks. A two-stage joint optimization task offloading strategy based on DRL and transfer learning is proposed to enhance resource utilization in VEC.

D. Summary

By analyzing the aforementioned literature, many existing studies fail to provide near-real-time services in highly dynamic VEC scenarios. The primary reasons are the high-speed mobility of vehicles causing unstable link transmissions, the overly large offloading decision space leading to decision algorithms not converging, and the varying emphasis on delay and energy consumption in different scenarios. In response to these issues, this paper combines DT and WGAN to pre-process vehicle historical data, aiming to predict future vehicle trajectories. This approach abandons traditional vehicle mobility models, reducing the decision space while being more in line with real-world scenarios. By using DIESEL for task offloading decisions and integrating the proposed adaptive model, we can provide real-time services for task offloading in any scenario without the need for extensive and complex training.

III. System Model and Problem Formulation

First, we present the system overview of the DT-empowered VEC network. The overall architecture of system network includes the transmission model and the analytic model. Then, the problem formulation is proposed based on an adaptive relationship model. The frequently used notations are listed in Table I.

A. Overview of DT-Empowered VEC Network

Fig. 1 illustrates the DT-empowered VEC network, which comprises three layers. We individually introduce each layer of the DT-network architecture and their specific applications.

1) Physical VEC Network Layer: This layer presents a scene with RSUs and vehicles with different densities and speeds. Traffic congestion leads to a closer distance between vehicles, and the speed of vehicles is relatively low. Every day, 1 TB of raw data on road conditions, driver behavior, and onboard sensor data is generated by the sensors and applications installed in vehicles. Using raw data extracted from an onboard sensor, the traffic condition is monitored in real time. RSUs are deployed in proximity to vehicles and provide radio interfaces within their instant wireless communication.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

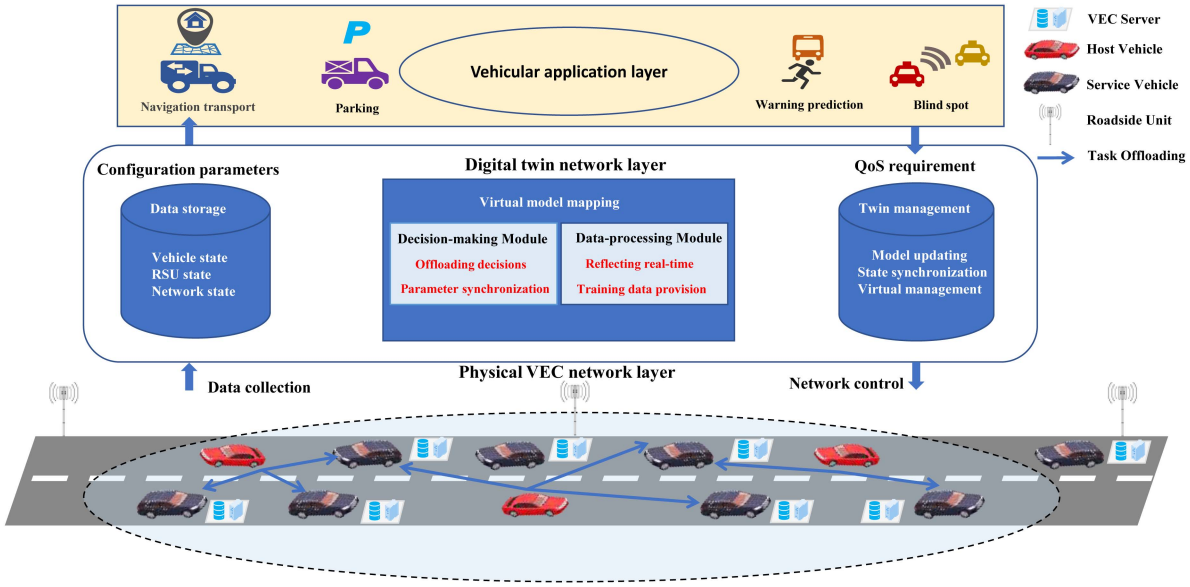


Fig. 1. DT-empowered VEC network.

TABLE I
TABLE OF SYMBOLS

Notation	Description	Metric
$\mathcal{N}$	The number of tasks	
$\mathcal{M}$	The number of service vehicles (SVs)	
D_i	Input size of task	MB
C_i	Required CPU cycles	Gcycle/s
T_i	Maximum torlerable delay	ms
$R^U_{i,j}$	The uplink data rate	Mbps
$R^D_{i,j}$	The downlink data rate	Mbps
B	Channel bandwidth	MHz
g_{ij}	Channel gain from vehicle v_i	
P^U	Transmission power of uploaded data	W
P^D	Transmission power of downloaded data	W
N_0	Noise power	dBm/Hz
t^{loc}_i	Local computation delay of HV_i	ms
f_0	Local computing power of the vehicle	Gcycle/s
E^{loc}_i	Local energy consumption	J
t^u_{ij}	The upload delay of HV_i	ms
t^d_{ij}	The download delay of HV_i	ms
t^{comp}_{ij}	The computing delay of of HV_i	ms
f_i	Computing power of the vehicle i	Gcycle/s
E^{off}_{ij}	Offloading energy consumption	J
S	The area of road covered for RSU	m^2
S^*	The average floor space of vehicle	m^2
p_s	The number of SVs in S	
p_h	The number of HVs in S	
p_{total}	The number of all vehicles in S	
f^{max}_i	The max computing power of vehicle i	Gcycle/s
F^{avg}_{log}	The average computing power of HV	Gcycle/s
F^{avg}_{off}	The average computing power of SV	Gcycle/s

2) Digital Twin Network Layer: This layer consists of three essential elements: data storage, virtual model mapping, and twin management. The data storage retrieves information from the physical VEC network layer and stores three distinct states: vehicle, RSU and network. The virtual model mapping is comprised of two primary elements: a decision-making module and a data-processing module. Decision-making module authorizes vehicles to offload their tasks to the surrounding RSUs while performing parameter synchronization. Data-processing module predicts the vehicle trajectory within the RSU communication range to achieve real-time response and further assist the vehicle offloading decision-making. Since implementing a mapping of the physical network is a key feature of the DT, the network layer of the DT needs a special project to meet this feature. Twin management is in charge of managing the virtual twin items at DT network layer. Therefore, the DT network layer provides network control feedback to the physical network layer via twin management.

3) Vehicular Application Layer: This layer can support a variety of vehicle applications based on the VEC physical network and DT network, including intelligent navigation, parking, accident warning prediction, and blind spot visual field monitoring. The DT network layer transmits stored structural parameters to the vehicular application layer. The function of data storage is to collect and store diverse network data for prediction, optimization, and virtual modeling. The vehicular application layer simultaneously sends Quality of Service (QoS) requests to the DT network layer. Accordingly, the DT network layer provides an effective and reliable computing service for the vehicular application layer.

B. Transmission Model

The transmission model is depicted in Fig. 1. RSU is deployed as an algorithm platform to make offloading decisions. There are many Host Vehicles (HVs) in lane that require task computation, and Service Vehicles (SVs) with computational resources are generated at random around the HVs. Within its range of communication, each SV has access to a server that can handle any low-delay processing that might be required. We suppose there are N total computation tasks in HVs, each of which is indexed by $\mathcal{N} = \{1, 2, \ldots, N\}$. The set of tasks is portrayed as

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

$\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_N\}$. $\mathcal{SV} = \{SV_1, SV_2, \ldots, SV_M\}$ denotes the group of service vehicles. Moreover, we use an exhaustive task model to describe every computational task. The task is denoted by the triplet of parameters, denoted as $\mathcal{Q}_i = (D_i, C_i, T_i)$, where D_i denotes the task offloading input size, C_i denotes CPU cycles to compute the i-th task, and T_i represents the delay of maximum tolerable.

The Shannon formula can calculate the data transmission rate based on the usual transmission model. However, the channel gain fluctuates at various times, resulting in an unpredictable data transmission rate. In particular, the offloading computation must rely on wireless data transmission, and the offloading computation for the design offloading scheme should be performed using dynamic methods that incorporate the wireless channel. According to Orthogonal Frequency-Division Multiple-Access (OFDMA), the rate at which tasks are uploaded from HVs to SVs employing OFDMA can be defined as follows:

$$R_{i,j}^{U} = \log_2\left(1 + \frac{P^U |g_{ij}|^2 l_m^{-\beta}}{N_0 + I_m^U}\right) \tag{1}$$

During the transmission of the i-th task, we use g_{ij} to describe the channel gain from HV to server j. P^U is uplink transmission power, l_m denotes the distance from HV to SV, β is the exponent of path loss, and I_m^U is the intercell interference of the uplink and N_0 is the spectral density of noise power at the receiver. The downlink transmission rate can be expressed similarly to (1):

$$R_{i,j}^{D} = \log_2\left(1 + \frac{P^D |g_{ij}|^2 l_m^{-\beta}}{N_0 + I_m^U}\right) \tag{2}$$

If too many tasks are offloaded to the SVs, it can lead to critical interference and affect communication efficiency. Moreover, interference in link of communication might lead to variations in the range of V2V communication. The instant V2V communication range $\mathcal{D}$ is represented as below:

$$\mathcal{D} = \mathcal{D}_0 \times \ell\left(1 + \frac{P^U |g_{ij}|^2 l_m^{-\beta}}{N_0 + I_m^U}\right) \tag{3}$$

where $\mathcal{D}_0$ is the initial transmission distance and ℓ is the adjustment coefficient.

C. Analytic Model

The HV dynamically selects candidate servers for task execution to optimise the performance of the total cost of the task. For the sake of clarity, Q_i is defined as residing on a server connected to the respective vehicular index. The computing capacity of the server can be expressed as $H = \{h_0, h_1, h_2, \ldots, h_M\}$, where h_0 denotes the capacity of the local computation.

1) Local Computing Model: When HVs execute local tasks on the road, they only use the resources of the vehicle itself for computation. The computation delay for local computation is given by (4), where C_i denotes the required CPU cycles and f_0 denotes the computing power of the vehicle i.

$$t_i^{loc} = \frac{C_i}{f_0} \tag{4}$$

The local energy consumption coefficient is denoted by γ. Once that is known, we can determine how much power is needed to perform local processing tasks.

$$E_i^{loc} = \gamma f_0^2 C_i \tag{5}$$

λ is tuning parameter, and it is assumed that λ^T and $\lambda^E (0 \leq \lambda^T, \lambda^E \leq 1)$ are relative delay and energy. Then, we can express the overall energy use for local processing as

$$C_{i0} = \lambda^T T_i^{loc} + \lambda^E E_i^{loc} \tag{6}$$

2) Edge Computing Model: To transfer computation tasks to the SVs, HV first transmits the tasks over a wireless channel to the selected VEC server, which incurs additional transmission overhead in comparison to local computation. According to the analysis of channel rate, the transmission delay necessary for the vehicle to upload the data of task Q_i to VEC server j is

$$t_{ij}^{u} = \frac{D_i}{R_{ij}^U} \tag{7}$$

where D_i denotes the size of task i and R_{ij}^U denotes the uplink date rate. The delay incurred when processing computation task Q_i on VEC server j is shown as follows:

$$t_{ij}^{comp} = \frac{C_i}{f_i} \tag{8}$$

t_{ij}^{d} is determined as follow, where ϖ is the proportion of output to input data size, where $\varpi \ll 1$. Therefore, the delay of the downlink is often ignored.

$$t_{ij}^{d} = \frac{\varpi D_i}{R_{ij}^D} \tag{9}$$

We define the decision set $A = \{a_{i0}, a_{i1}, \ldots, a_{ij}\}$. Based on (4), (7), (8), and (9) above, we use t_{ij}^{off} to represent the total delay for the whole task offloading in (10).

$$t_{ij}^{off} == \begin{cases} t_i^{loc} & \text{if } a_{ij} = 0 \\ t_{ij}^{u} + t_{ij}^{comp} + t_{ij}^{d} & \text{if } a_{ij} = 1 \end{cases} \tag{10}$$

$$T_{ij}^{off} = \max\left\{t_{i1}^{off}, t_{i2}^{off}, \ldots, t_{ij}^{off}\right\} \tag{11}$$

T_{ij}^{off} is the collection of t_{ij}^{off}, indicates the maximum offloading delay.

Rather than processing the task on the local server, offloading to the SVs requires additional energy for sending and receiving. Given the received power P^d, the formula for energy consumption is as below:

$$E_{ij}^{off} == \begin{cases} E_i^{loc} & \text{if } a_{ij} = 0 \\ P^u t_{ij}^{u} + P^d t_{ij}^{d} + f_i^2 C_i & \text{if } a_{ij} = 1 \end{cases} \tag{12}$$

Based on (11) and (12), for the given server j and the given task i, the overall overhead of the edge execution strategy is

$$C_{ij} = \lambda^T T_{ij}^{off} + \lambda^E E_{ij}^{off} \tag{13}$$

where the values of λ^T and λ^E are determined in the adaptive model in the next subsection.

D. Adaptive Relationship Model

In this subsection, we design an adaptive relational model to define the relationship between the weights λ^T and λ^E based on

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

the QoS and energy influence factors. The relationship between λ^T and λ^E is shown as follows:

$$\lambda^T + \lambda^E = 1 \tag{14}$$

where λ^T is the weight of QoS and λ^E is the weight of energy efficiency. The idea of the adaptive model is proposed by coordinating these two weights dynamically. The above parameters are collected from the physical units by DT, and the system objectives are adjusted by the model. The adjustment of weights is mainly guided by the following criteria:

- *Consideration of overall QoS:* The first premise is QoS, with $\varepsilon \in \{0, 1\}$ indicating the weight of the influencing factor. We develop the concept of traffic density ε, which is based on the coverage of the RSU and the total number of HVs and SVs. In (15), S is the area of the road covered by RSU. S^* is the area of one vehicle. For example, in dense scenarios such as urban rush hour traffic, vehicle users require much higher QoS than energy requirements, while in remote areas where vehicles are more sparse, vehicles need enough loading space and computing power to handle long-distance offloading tasks.

$$\varepsilon = 1 - \frac{S - S^*}{S} \tag{15}$$

- *Evaluation of load of SV:* The load degree is mainly influenced by relationship between the SVs and the number of vehicles' computing power to determine the weight of the factor $\delta \in \{0, 1\}$, which is shown in (16). k is a parameter that controls the steepness of the sigmoid curve. L represents the load degree of SV, which can be quantified based on factors such as CPU utilization or task queue length. $L_{\text{threshold}}$ is a threshold value that determines when SVs are considered to have a high load degree, and it can be set based on the requirements of the system. This equation models δ as a sigmoid function, where higher values of L relative to the threshold result in higher values of δ, so the higher the QoS requirement, the higher the load situation requirements.

$$\delta = \frac{1}{1 + \exp(-k(L - L_{\text{threshold}}))} \tag{16}$$

- *Comparison of computing power:* We assume that the local computing power of the HV is lower than SV, and the weight $\eta \in \{0, 1\}$ of this influence factor is determined by comparing the average computing power between HVs and SVs. The higher the QoS requirements, the higher the computing power requirement of the SVs. δ and η are juxtaposed influences which satisfy $\delta + \eta = 1$.

Taking into account the above principles, we can summarize the final formula in relation to the weight λ^E in (17).

$$\lambda^E = \varepsilon \left\{ \delta \left(1 - \frac{\sum_{i=1}^{p_s} f_i^{max}}{\sum_{i=1}^{p_{total}} f_i^{max}} \right) + \eta \frac{F_{loc}^{avg}}{F_{off}^{avg}} \right\} \tag{17}$$

where p_s is the number of SVs and p_{total} is the sum of the number of two types of vehicles. f_i^{max} is the maximum computing power of the vehicle i. F_{loc}^{avg} is the average computing power of the HV when the vehicle is locally offloaded. F_{off}^{avg} is the average computing power of the SV when the SV offloads tasks from the HV. The relationship between the number of SVs and HVs is affected by multiple factors. With the number of SVs decreases, the values of ε, δ, and η increase, the percentage of λ^E increases, so the factor of energy has a larger influence in this situation.

E. Problem Definition

In this subsection, according to the delay and energy consumption analysis in (13), we can use (11) to consider the maximum delay of the schemes, including local computation, full offloaded computation, and local with offloaded computation. Our goal is to minimize ϕ. Based on the preceding evaluation of parameters, an optimization problem for the minimization of the cost of HV for the processing of all tasks is shown as below:

$$\mathcal{P}_1: \quad \min_{\mathcal{A}, f_i} \quad \phi = \sum_{i=1}^{N} \sum_{j=0}^{M} a_{ij} C_{ij} \tag{18}$$

$$\text{s.t.} \quad \sum_{j=0}^{M} a_{ij} = 1, \forall i \in \mathcal{N} \tag{18a}$$

$$a_{ij} \in \{0, 1\}, \forall i \in \mathcal{N}, \forall j \in \mathcal{M} \tag{18b}$$

$$T_{ij}^{off} \leq T_i^{max}, \forall i \in \mathcal{N} \tag{18c}$$

$$0 \leq f_i \leq f_i^{max}, \forall i \in \mathcal{N} \tag{18d}$$

$$\sum_{j=0}^{M} a_{ij} f_i \leq f_i^{max}, \forall i \in \mathcal{N} \tag{18e}$$

In (18), the objective function is to minimize the weighted sum of all N tasks' total cost, where (18a) represents every computation that can only combine one server. In addition, we provide the offloading decision profiles as follows: $\mathcal{A} = \{a_{ij} \mid i \in \mathcal{N}, j \in \mathcal{M} \cup \{0\}\}$. (18b) means that a_{ij} is the binary signal indicating if the i-th task is assigned to server j for computation. According to (18c), each task must be completed within the maximum time T_i. (18d) implies that the allocation of resources from other vehicles to HV is positive and cannot exceed the total computing resources. (18e) states that when vehicle i receives j tasks, the sum of the allocated computing resources is less than the total computing resources.

According to (13), (17) and (18), ϕ is nonlinear relative to $\mathcal{A}$ and f_i [37]. Consequently, the issue is a constrained Mixed-Integer Nonlinear Program (MINLP) with an NP-hard solution complexity. No polynomial-time algorithms are currently accessible for the solution of MINLP exponential explosion problem since it has been established. In order to get a handle on the issue, meta-heuristic optimisation algorithms are employed in this work.

IV. Proposed Solution

In this section, STRIVE is introduced in detail, which can be presented in two aspects. First, we propose the mapping process of STRIVE, using a modified version of WGAN as the preprocessing process of STRIVE, which is an effective assistant

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

for the DIESEL scheme. Second, the proposed overall STRIVE scheme is built upon the advanced model of DT.

A. Mapping Process of STRIVE

We first introduce the pre-processing process based on DT. We construct the overall DT architecture of the system through a traffic prediction model. By combining DT with WGAN, we map the constructed prediction model to physical entities, providing effective assistance for task offloading decisions.

1) Preliminaries: GAN is a traditional generative model. It utilizes only backpropagation and does not require complex Markov chains, unlike other generative models such as GSNs and Boltzmann machines [38]. Both the generator and the discriminator work to improve their respective abilities to distinguish between real and generated data. However,traditional GAN is susceptible to pattern collapse and gradient disappearance. Since the gradient update of the generator comes from discriminator, the training level of generator and discriminator must be carefully considered during the training process, and if the generated samples are not very realistic while discriminator gives a correct evaluation, then the generator will consider that the generated results are correct, so that both parties will engage in self-deception, resulting in missing information and incomplete features in the generated results.

The currently proposed WGAN can solve the above problem. WGAN applies the Wasserstein distance due to its superior smoothing properties compared with kullback-leibler (KL) scattering and JS scattering, theoretically solving the gradient vanishing problem. The problem is demonstrated in detail in IV-A3. Using mathematical transformations, we approximate the Wasserstein distance and then optimize it using a discriminator neural network with a constrained parameter space. By eliminating training instability, WGAN provides a reliable indicator of the training process, which has a direct bearing on the quality of the output samples.

2) Data Extraction Based on DT: In this subsection, we build the DT network layer to finish the data extraction based on how the raw data will be mapped in the future. When a vehicle moves on the road, the neighbouring vehicles on the same road have an impact on the movement state of the vehicle. We consider the case of a two-way road section and model the road section and the vehicle separately. Our objective is to build a model of the DT network to predict the future traffic state of the road based on its current state. In the DT network model, the set of driving states is defined as follows:

$$V = \{V_c \cup V_r\} \tag{19}$$

where V_c and V_r denote the sets of vehicle state and road state, respectively. The vehicle state V_c is defined as follows:

$$V_c = \{v_{id}, x, y, v_s\} \tag{20}$$

In (20), v_{id} is the number ID of vehicle, (x, y) reflects the location of vehicle, and v_s represents the current speed at time t. The set of road conditions can be defined as follows:

$$V_r = \{P_s, P_e, p_f, p_d, p_v, N\} \tag{21}$$

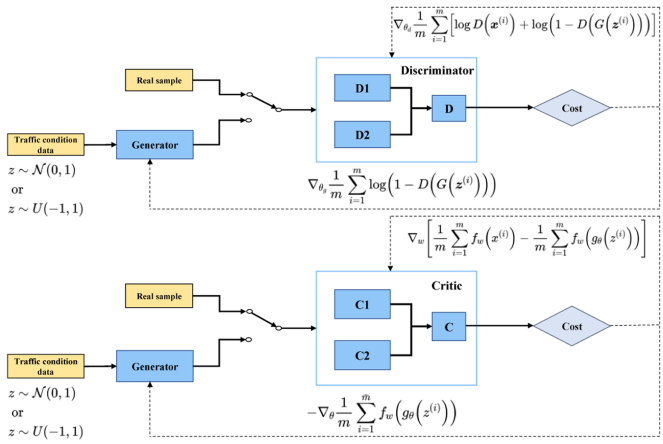


Fig. 2. System Framework Diagram of GAN and WGAN.

In (21), P is a two-dimensional coordinate point, and P_s and P_e represent the start and end points of the road. p_f indicates if there is another vehicle driving ahead, p_d indicates the gap between the vehicle and the one in front, and p_v is the vehicle's speed. N denotes the number of vehicles at current time.

3) Prediction Model of WGAN: Next, we introduce the prediction model for vehicles by using WAGN for trajectory prediction. To solve the input-output mapping function, we employ a neural network model for traffic forecasting in the future. Here we detail the advantages of WGAN over traditional GAN.

The current proposed WGAN can solve the above problem. WGAN replaces the Jensen-Shannon (JS) divergence with the Wasserstein distance, which is defined below:

$$W\left(U_r, U_g\right) = \inf_{\gamma \sim \Pi(U_r, U_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|] \tag{22}$$

In (22), $\Pi(U_r, U_g)$ is the collection of all potential joint distributions of the combination of U_r and U_g. For each potential joint distribution γ from which one may sample $(x, y) \sim \gamma$ from it to obtain a real sample x and a sample generated from y, and compute the distance of the sample $\| x - y \|$. In turn, the expected distance between sample pairs $\mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|]$ is calculated. The Wasserstein distance is the lower bound that can be placed on this expectation among all possible joint distributions.

By using a neural network, WGAN solves the problem of using Wasserstein distance with traditional GAN. This algorithm is briefly described in (23).

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x\sim\mathbb{U}_r}\left[f_w(x)\right] - \mathbb{E}_{z\sim p(z)}\left[f_w\left(g_\theta(z)\right)\right] \tag{23}$$

$\{f_w\}_{w\in\mathcal{W}}$ is the parameter that you need to optimize. The neural network is extremely similar to the discriminator in GAN. Here we name it critic to distinguish discriminator.

The system framework diagram of GAN and WGAN is shown in Fig. 2. The critic rejects the sigmoid function in the last layer because its output is a general score rather than a probability, and the objective function lacks a logarithmic term. To address the above problem, weight clipping is introduced as a method to truncate parameters within a particular range after each update.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

This approach helps stabilize the learning process and prevents the critic from giving overly optimistic or pessimistic estimates of future rewards. Therefore, WGAN no longer needs to consider how to balance the training degrees, which avoids the phenomenon of mode collapse and reflects excellent robustness. While the loss of critic properly represents the quality of samples created by the generator, it may also be utilized as a qualitative metric of GAN training success.

Therefore, in this paper, WGAN is employed to train models and predict vehicle trajectories. Accordingly, our model represents the trajectory prediction problem as a time-varying prediction method. We provide the following definition for the mapping function of trajectory prediction for vehicles by using DT:

$$V_t = F\left(V_{t-1}\right) \tag{24}$$

In (24), V_t denotes the vehicle's driving state during the previous time interval. Using the vehicle state data predicted after one second as input, the next second's vehicle state is predicted. The prediction is a continuous iterative process that is not terminated until the desired iteration of vehicle trajectory data is obtained. This way has the benefit that the final predicted trajectory of the vehicle's accuracy can be high by simply minimizing the error of each prediction. Next, the generative and critic models are presented separately.

1) *Generative Model:* The generative model of WGAN is similar to the traditional GAN. The task of predicting vehicle trajectories using WGAN can be viewed as teaching the mapping function G based on the driving conditions of the vehicle. In lines 5–9 of Algorithm 1, the following generative training procedure is illustrated. V_{tg} represents the vehicle's state set as generated by the generative model, which is denoted as $V_{tg} = G(V_{t-1})$. V_{t-1} indicates driving condition at time $t-1$. The generated V_{tg} is combined with the prevailing V_{t0} to obtain the vehicle V_t. V_t is the generated model's input, and the result represents a forecast of the vehicles' state at the next time, which is denoted by V_{t+1}. Multiple iterations can produce continuous vehicle trajectory data.

2) *Critic Model:* We use the critic model to distinguish the discrimination model in GAN. The critic's responsibility is to make a distinction between real data and vehicle trajectory data. If the generated and actual trajectories are quite similar to one another, then the critic result will be extremely close to 1. The critic is unable to evaluate the generator's output based solely on V_{t-1} or V_{tg}. Therefore, the generator's output data on vehicle trajectories cannot be used as input by the critic to check for accuracy. The input of the generator is necessary for the critic to evaluate how close the prediction of the generator is to the actual outcome. The identifying method is as follows:

$$\Gamma = C\left(V_{t-1}; G\left(V_{t-1}\right)\right) \tag{25}$$

In (25), Γ represents the probability that a critic can identify produced data from genuine data. It is evident from the preceding equation that the critic must accept both sets of data to determine the validity of the mapping function. The first section is the generator's input data, which includes traffic state data, and the next section is the generator's output data. As demonstrated in Fig. 2, $G(V_{t-1})$ is the data of prediction, and V_{t-1} is the data of driving condition. Therefore, for the critic model, $C1$ and $C2$ make up the discriminant network. Both $C1$ and $C2$ in Fig. 3 are three-layer deep neural networks that influence how probable it is that a critic will be able to tell that the generated data is not real. For the goal of teaching the critic, both positive and negative examples are used. In Algorithm 1, lines 11–14, Simulation of Urban Mobility (SUMO) simulator-generated vehicle trajectory data constitutes the positive samples. In contrast, the negative samples are the data predictions generated with reference to the input by generator. According to the positive and negative samples, the critic awards the generator.

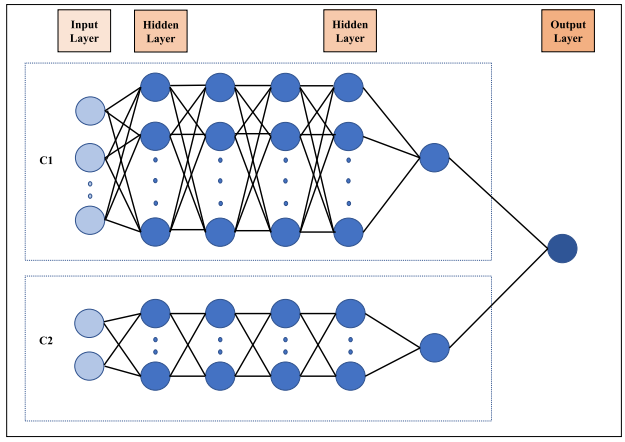


Fig. 3. Structure of the critic model.

To conclude, Algorithm 1 shows all the specifics of the WGAN. G_ς and C_τ are trained alternatively after initializing the generator and the critic. When the generator is improved by training with G-step updates, the critic must be retrained to keep up with the improved speed of the generator. Positive and negative samples are generated from the current data set by the generator while training the critic. To increase the precision of the generative model, it is advisable to marginally increase the learning rate of the generator in comparison to the critic. It causes the generator to converge more quickly than during normal training.

In this way, we use DT to map the future condition of the road and vehicle to physical entities and use WGAN to simulate the actions of drivers to independently predict the vehicle's condition. We build the DT model based on the future mapping of raw data by constructing the DT network layer, thereby reducing the V2V offloading decision space, which can be defined as an integral part of the data pre-processing.

B. Offloading Scheme With Improved Genetic Simulated Annealing-Based Particle Swarm Optimization

In this subsection, based on the DT model established by the future trajectory data of the vehicles, we introduce a new swarm intelligence algorithm to solve the offloading decision problem called DIESEL. It combines the advantages of the powerful local search capability of PSO and the stochastic global search

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Mapping Model of WGAN.

Input: the data set $\mu = \{Z_{1:i}\}$, Generator G_ς, Critic C_τ, the learning rate for Generator and Critic ω_g and ω_c
Output: mapping function G_o

Initialize G_ς, C_τ with random weight ς, τ;
$Z_t = \{V_{t-1} \cup V_{tg}\}$;
$o \leftarrow \varsigma$;
repeat
  for *G-processing* **do**
    Generate sample $\hat{Z}_t = \left\{V_{t-1} \cup \hat{V}_{tg}\right\}$;
    $\hat{V}_{tg} = G_\varsigma\left(V_{t-1}\right)$;
    $\nabla = C_\tau\left(\hat{Z}_t\right)$;
    $\varsigma \leftarrow \varsigma - \omega_g \nabla$;
  end
  for *C-processing* **do**
    Generate sample $\hat{Z}_t$ by using current G_ς;
    $\nabla = C_\tau\left(Z_t\right) - C_\tau\left(\hat{Z}_t\right)$;
    $\tau \leftarrow \tau + \omega_d \nabla$;
  end
  $o \leftarrow \varsigma$;
  return o;
until *WGAN converge*;

capability of GA. In addition, DIESEL has the Metropolis acceptance rule in SA, where the annealing coefficient k is determined by real-time state, and the annealing efficiency is dynamically adjusted by DT's collection of raw data from physical entities, making the whole system more suitable for VEC scenarios with high dynamics.

1) Preliminaries: PSO belongs to a type of evolutionary algorithm that originated from the study of the predatory behaviour of bird flocks [39]. Its fundamental concept is to find the optimal solution through collaboration and information sharing among population members, find the optimal solution with the highest fitness through multiple iterations, and find the global optimal solution by following the currently searched optimal solution.

GA is a mathematical formula for the biological evolutionary process that replicates the mechanism of natural selection and the genetics of Darwinian biological evolution. It is a technique for finding optimal solutions by modelling the natural evolutionary process [40].

Simulated Annealing Algorithm (SAA) is a general probabilistic algorithm built on the solid annealing principle [41]. It begins with a high initial temperature and a falling temperature parameter and combines several probabilistic jumping characteristics to randomly search the solution space for the optimal global solution of the objective function.

Among the several common search algorithms, PSO converges quickly but is prone to falling into local optima when solving constrained problems with complex solution spaces. While the operation of GA provides higher individual diversity and improves the robustness of global search. By using Metropolis acceptance rule, SA not only converges slower, but it can also jump from the local optimum to global optimum by choosing the optimal cooling temperature rate. Therefore, first, we combine the benefits of the three aforementioned algorithms. Second, the weighing parameters are modified in real-time to adapt to the dynamic VEC scenario.

2) Problem Transformation: In (18), ϕ is a nonlinear of $\mathcal{A}$ and f_i. We employ a penalty function method to manage a large number of complex constraints [42]. Then, the constrained adaptive cost minimization problem is converted into an unconstrained adaptive cost minimization problem before being solved by the DIESEL algorithm. Let $\mathbf{k}$ represent the vector of decision variables containing the parameters listed above.

$$\min_{\mathbf{k}} \quad \left\{\tilde{\phi} = \varpi P(x) + \phi\right\} \tag{26}$$

where $\tilde{\phi}$ represents the transformed objective function, and ϖ represents a positive constant approaching infinity. The total penalty for the whole unsatisfied constraints is defined as $P(x)$. The formula is shown below:

$$P(x) = \sum_{m=1}^{\varpi^{\leqslant}} \left(\max\left\{0, -g_m(\mathbf{k})\right\}\right)^{\overset{1}{\upsilon}} + \sum_{n=1}^{\varpi^{=}} \left|h_n(\mathbf{k})\right|^{\overset{2}{\upsilon}} \tag{27}$$

In (27), $\varpi^{\leqslant}$ and $\varpi^{=}$ represent the number of inequality and equality constraints, respectively. $\overset{1}{\upsilon}$ and $\overset{2}{\upsilon}$ are the positive numbers. Every inequality constraint $m(1 \leq m \leq \varpi^{\leqslant})$ is converted into $g_m(\mathbf{k}) \geq 0$. Its penalty is $\left(\max\{0, -g_m(\mathbf{k})\}\right)^{\overset{1}{\upsilon}}$ while it is violated. Otherwise, the result is 0. Every equality constraint $n(1 \leq n \leq \varpi^{=})$ is converted into $h_n(\mathbf{k}) = 0$. Its penalty is $\left|h_n(\mathbf{k})\right|^{\overset{2}{\upsilon}}$ if it is violated. For instance, (18c) is first turned into $T_{ij}^{off} - T_i \leq 0$, and its penalty is $\left(\max\{0, -(T_{ij}^{off} - T_i)\}\right)^{\overset{1}{\upsilon}}$. Then, the resulting unconstrained problem is further solved with the proposed DIESEL.

There are some exact and typically conclusive algorithms to solve the unconstrained problem, e.g., dynamic programming [43] and conjugate gradient descent [44]. However, they rely excessively on the structure of the objective function and cannot comprehensively solve such unconstrained problems. Current RL methods require multiple trainings for the complex and dynamic scenarios in VEC. Additionally, when faced with a large decision space due to numerous tasks, these methods might not converge. Several studies now opt for typical meta-heuristic optimization algorithms with high robustness, simple implementation, and the capability to rapidly handle discontinuous and nonlinear problem characteristics. They are widely used for their ability to solve a variety of complex and difficult problems without drawbacks of deterministic algorithms.

3) Solution of DIESEL: In PSO, the particles in the swarm use the particles and the overall learning experience to change their position and velocity. $\tilde{\mathbf{X}}$ represents the size of swarm. v_i and x_i represent the velocity and location for every particle $i(i = 1, 2, \ldots, \tilde{\mathbf{X}})$. $\mathbf{N}^d$ represents each position's dimension. All the elements store $\mathcal{A}$ and f_i. Therefore, $\mathbf{N}^d = 2M + 1$. We let x_i^l and x^g represent the locally and globally optimal positions,

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

respectively. In a word, the formulas for v_i and x_i are shown below:

$$v_i = \omega_0 v_i + c_1\omega_1\left(x_i^l - x_i\right) + c_2\omega_2\left(x_g - x_i\right) \tag{28}$$

$$x_i = x_i + v_i \tag{29}$$

In (28), ω_1 and ω_2 represent two randomly generated constants in $(0, 1)$ and the inertia weight is ω_0. c_1 and c_2 denote individual and societal acceleration coefficients, which both reflect the influence of x_i^l and x_g. If x_i^l and x_g differ significantly, premature convergence will happen and the process of optimization will oscillate. Genetic operations in GA can produce better particles and enhance the global search capabilities of PSO to deal with this problem. Therefore, $\widetilde{x}_i$ represents the position of a superior particle assigned to every particle i. In our algorithm, $\widetilde{x}_i$ is constructed by combining x_i^l and x_g. The formula for $\widetilde{x}_i$ is shown below:

$$\widetilde{x}_i = \frac{c_1\omega_1 x_i^l + c_2\omega_2 x^g}{c_1\omega_1 + c_2\omega_2} \tag{30}$$

Therefore, the value of v_i and x_i are shown as below:

$$v_i = \omega_0 v_i + c_3\omega_3\left(\widetilde{x}_i - x_i\right) \tag{31}$$

$$x_i = x_i + v_i \tag{32}$$

In (31), c_3 represents the acceleration coefficient for every superior particle, and ω_3 is a random number vector uniformly generated from 0 to 1.

Besides, the date must be coded in GA, and the position or velocity of x_i^l and x^g is composed of a string of binary bits. So we denote the mutation possibility by using c_4. For x_i^l and x^g, single-point crossover is performed to produce x_i^l as an offspring. Each bit of x_i^l undergoes the mutation operation with the current probability c_4. By utilizing this algorithm, it is possible to avoid early convergence to a local optimum. Furthermore, for each particle i, the search is directed by one of two possible choices: either x_i^l or $\widetilde{x}_i$.

Next, the improvement process of the DT-based adaptive simulated annealing algorithm is described. Based on the following guidelines, if $\widetilde{\phi}(x_i^l) \le \widetilde{\phi}(\widetilde{x}_i)$, X_I^l is selected as the best particle for every particle i, otherwise, the best particle is $\widetilde{x}_i$. Thus, an optimal particle is selected for each particle. This method determines a parent particle for each particle.

Let x_i^k and x_i^{k+1} represent the positions of particles i in iterations k and $k+1$. Then, the formula for x_i^{k+1} is shown below:

$$v_i = \omega_0 v_i + c_3\omega_3\left(\widetilde{x}_i - x_i^k\right) \tag{33}$$

$$x_i^{k+1} = x_i^k + v_i \tag{34}$$

If $\widetilde{\phi}(x_i^{k+1}) \le \widetilde{\phi}(\widetilde{x}_i^k)$, x_i^{k+1} is selected; otherwise, it is conditionally selected if

$$\exp\left\{\frac{-\left(\widetilde{\phi}\left(x_i^{k+1}\right) - \widetilde{\phi}\left(x_i^k\right)\right)}{\mathbf{K}T}\right\} > \omega_4 \tag{35}$$

Algorithm 2: Improved Genetic Simulated Annealing-Based Particle Swarm Optimization Algorithm.

1 Randomly initialize each particle's velocity and position;
2 Initialize c_1, c_2, c_3, ω_0, ω_0^{max}, ω_0^{min}, ω_1, ω_2, ω_3 and $\widetilde{\mathbf{X}}$ of PSO, ω_4 and T of SA, c_4 of GA, and k_{max}, ω_5 and ω_5^{max};
3 Update fitness values $\widetilde{\phi}$ of particles with (26);
4 Calculate x_i^l and x^g with (33) and (34);
5 Set $k = 1$;
6 While $k \le k_{max}$ and $\omega_5 \le \omega_5^{max}$;
7 Perform crossover on x_i^l and x^g to produce an offspring $\widetilde{x}_i$;
8 Perform mutation on each row of offspring to obtain the offspring $\widetilde{x}_i$;
9 Perform selection to specify whether x_i^l or $\widetilde{x}_i$ is selected;
10 Update velocities of particles by (33);
11 Update positions of particles with SA's Metropolies acceptance rule in (34), (35) and (36);
12 Using (26) to calculate fitness value $\widetilde{\phi}$ of particles;
13 Update the locally optimal position x_i^l of each particle i;
14 Update the globally position x_g of the population;
15 Update $\mathbf{K}$ by using (36);
16 $\omega_0 \leftarrow (\omega_0^{max} - \omega_0^{min})\frac{k_{max}-k}{k_{max}} + \omega_0^{min}$;
17 Update percentage ω_5 of particles with the same $\widetilde{\phi}$;
18 end while;
19 return x^g

where ω_4 denotes a uniformly generated random number between $(0, 1)$, T represents temperature during iteration k and $\mathbf{K}$ is the annealing adaptation factor, which follows the following rules:

$$\mathbf{K} = (1 - \varepsilon)\,\mathbf{K} \tag{36}$$

We set the annealing adaptive factor $\mathbf{K}$ to be dynamic, and the value of $\mathbf{K}$ is determined based on the real-time state of the system. DT is used to collect the original data of physical entities and adjust annealing efficiency dynamically. As the temperature decreases, the temperature attenuation coefficient also decreases. ε is the environmental density in the adaptive model. For example, if the scenario requires higher QoS when the number of vehicles increases, the value of $\mathbf{K}$ decreases, which accelerates the rate of temperature reduction, lowers the length of the Markov chain, decreases the number of searches in the algorithm, and accelerates the convergence, thus ensuring a decrease in the required delay.

DIESEL incorporates the benefits of PSO, GA, and SA. It combines the genetic operations on x_i^l and x^g to generate better particles and updates the position of the particles by using the Metropolis of SA acceptance rules. Thus, the search functionality of PSO in DIESEL is enhanced.

Algorithm 2 demonstrates the pseudocode for DIESEL. Line 1 initializes the velocities and positions for particles with random values. Line 2 initializes c_1, c_2, c_3, ω_0, ω_0^{max}, ω_0^{min}, ω_1, ω_3, $\widetilde{\mathbf{X}}$ of PSO, the temperature's cooling rate ω_4, starting temperature

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

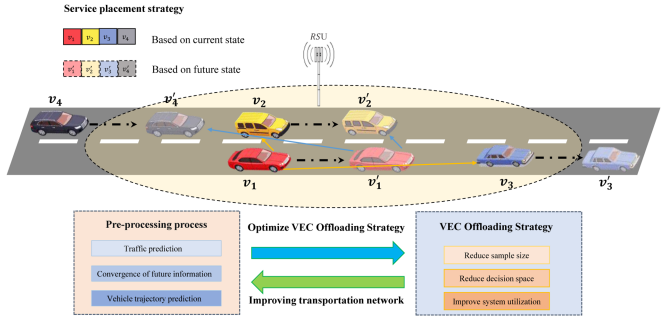


Fig. 4. Overall process of STRIVE scenario diagram.

T of SA, and the mutation possibility c_4 of GA. ω_0^{max} and ω_0^{min} represent the maximum and minimum inertia weight limits ω_0. k_{max} is the number of iterations. ω_5 represents the proportion of particles with the same $\widetilde{\phi}$. Line 3 updates the fitness values $\widetilde{\phi}$ by (26). Line 4 calculates x_i^l and x^g. Lines 7–9 show the process of crossover, mutation, and selection in GA. First, operate crossover on x_i^l and x^g with the process of single-point crossover, and then the offspring $\widetilde{x_i}$ is obtained by mutating with the probability c_4. The next step is the selection operation, where each particle i is assigned the value of x_i^l or $\widetilde{x_i}$. Line 10 modifies the particle velocities with (33). Line 11 modifies the particle positions by using Metropolies acceptance rule from (34), (35) and (36). Line 12 updates the fitness value $\widetilde{\phi}$ by (26). Lines 13 and 14 update the locally optimal position x_i^l and the global position x^g. Line 15 updates $\mathbf{K}$ by using (36) to adapt to dynamic scenarios. Line 16 reduces ω_0 linearly from ω_0^{min} to ω_0^{max}. Line 17 updates the proportion ω_5 of particles with the same $\widetilde{\phi}$. After transforming x^g into decision variables in line 19, the DIESEL gets our final results.

Furthermore, we give a complexity analysis of DIESEL. In Algorithm 2, the running cost of the algorithm is mainly generated through the while cycle. Assuming the worst-case situation, the cycle terminates after k iterations. In lines 5–18 of each DIESEL iteration, the time complexity is $\Theta(\mathbf{X} \mid \mathbf{N}^d)$. It is important to note that $\mathbf{N}^d = 2M + 1$. So in Algorithm 2, each iteration's time complexity is $\Theta(k \mid \mathbf{X} \mid 2M + 1)$.

By integrating the aforementioned three algorithms, we demonstrated the interpretability of DIESEL. First, using GA provides population diversity, powerful crossover and mutation operations to explore new solutions. Leveraging its features can assist PSO in conducting more refined searches. SA increases the broad exploration of the solution space by randomly accepting inferior solutions, aiming to avoid getting trapped in local optima, which further optimizes DIESEL.

C. Overall Process of STRIVE

In this subsection, we integrate the DT-based pre-processing process in IV-A and the adaptive environment offloading decision algorithm in IV-B to present the complete STRIVE offloading scheme in detail. On the one hand, broken links can further lead to an incomplete offloading task between RSUs and vehicles. On the other hand, the speed difference between vehicles can lead to incomplete task offloading, e.g., if the speed of the HV is too fast compared with the surrounding SVs, unstable transmission between vehicles due to excessive distance between vehicles. Therefore, we use the method of IV-A. By mapping DT to physical entities, the exact location of the vehicle at a future time can be calculated to determine whether it is within the RSU. By calculating the relative distance between the vehicles at a future time, we can determine whether stable communication can be established between the vehicles and whether the offloading process can be completed. As shown in Fig. 4, we assume that the position of vehicles v_1-v_4 at the passing of t^{off} seconds is $v_1{}'$- $v_4{}'$, v_1, v_2 and v_3 are in the RSU range, while v_4 is not in the RSU range. We can derive the relative distance $d(t)$ between $v_1{}'$ and $v_2{}'$ at the elapsed moment t. We assume that $d(t)_{max}$ is the maximum communication distance between vehicles that can be offloaded. It influenced by the communication model described in Section II-B. If it is satisfies (37), v_1 can offload to v_2; otherwise, it cannot.

$$d(t) \leq d(t)_{max} \tag{37}$$

Algorithm 3: STRIVE Task Offloading Scheme.

Input: VEC network
Output: The total consumption for local processing, the allocation of computing resources per vehicle and offloading decisions

1 Initializes the parameters by extracting the raw data from the VEC network;
2 Calculate the adaptive weight λ^E according to (17);
3 Calculate the relative distance of the vehicles and obtain the advanced DT model by executing Algorithm 1;
4 Calculate the allocation of computing resources of vehicles and offloading decisions by executing Algorithm 2

Algorithm 3 illustrates the entire STRIVE task offloading process, its input requires the entire VEC network and its output is the total consumption, the allocation of each vehicle's computing resources, and the offloading decisions. The overall execution process of STRIVE can be summarized as follows: First, STRIVE extracts and initializes parameters, including the number of RSUs, HVs and SVs. Second, according to the above-extracted parameters, calculate the adaptive weight λ^E by (17). Third, STRIVE calculates the future specific position and the relative distance of the vehicle per second by executing Algorithm 1 to establish the DT model. Fourth, STRIVE adjusts the adaptive annealing factor according to the advanced DT model. Finally, STRIVE executes Algorithm 2 to calculate the allocation of computing resources per vehicle and offloading decisions based on the adaptive weighing factors.

V. Performance Evaluation

This section provides an evaluation of the STRIVE's performance, which can be divided into four parts. We present the evaluation parameters and scenarios in the first part. The

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS

Parameter	Symbol	Value
Noise power density	N_0	-174 dBm/Hz
Bandwidth	B	10 MHz
Data upload power	P_u	0.1W
Data download power	P_d	0.1W
Channel gain	g_{ij}	$(10^{-8}, 10^{-7})$
Data size of each task	D_n	(0.1, 0.5) MB
Task required CPU cycle	C_n	(0.1, 0.5) $Gcycle/s$
Maximum delay tolerance	T_i^{max}	2s
Max computing power of the vehicle	f_i^{max}	$1.5 \times Gcycle/s$
The average compute power of HV	F_{loc}^{avg}	$0.3 \times Gcycle/s$
The average compute power of SV	F_{loc}^{off}	$0.4 \times Gcycle/s$
V2V communication radius	d_{max}	300m
The number of tasks	M	(5, 20)
Max inertia weight	ω_0^{max}	0.9
Min inertia weight	ω_0^{min}	0.4
The number of iterations	k_{max}	10^3
The coefficient of acceleration	c_1	0.5
The coefficient of social acceleration	c_2	0.5
The coefficient of superior particle	c_3	1.496
Mutation probability	c_4	0.1
Max Temperature	T	1000
The percentage of fitness value	ω_5	0.95
The coefficient of penalty function	$\tilde{\phi}$	10^4
The number of populations	$\mathbf{X}$	100

second part is the performance evaluation of the DT-based pre-processing models. Third, the adaptability of the STRIVE model is evaluated in different scenarios. Fourth, we evaluate the task-offloading effectiveness of STRIVE by comparing it with other cutting-edge task offloading schemes.

A. Evaluation Parameters and Scenarios

The data from our simulation experiment was generated by SUMO. The simulation platform was Python 3.7 under the Win 11 64-bit operating system, and the experiment was implemented using numpy, torch, matplotlib, and other libraries. The open source code for the platform has been uploaded and made accessible on Github.[1] To normalize the coordinates of all vehicles' topologies, we first train WGAN by using the vehicle coordinate transformation model. We take 2000 vehicles in the SUMO dataset as training samples, of which 1600 vehicles are training sets, 400 vehicles are test sets. To obtain vehicles that satisfy the offloading requirements and to accurately predict the precise future locations of the vehicles in each time slot. Table II lists the main simulation parameters. As for the communication model, we set the noise power density as $N_0 = -174\,\text{dBm/Hz}$, the channel gain g_{ij} between SV and HV varies with time between $(10^{-8}, 10^{-7})$. We set the upload power and download power as $P_u = 0.1\,\text{kW}$ and $P_d = 0.1\,\text{kW}$. To parameterize the task, the number of tasks is set to N from 5 to 20, the task's required CPU cycle C_n is set from 0.3 Gcycle/s to 0.5 Gcycle/s, and the data size of task D_n is set from 0.8 MB to 1.0 MB.

The typical meta-heuristic optimization algorithm is very sensitive to the parameters' setting. Therefore, according to the existing study [45], the design algorithm is applied to investigate the optimal combination of parameters for DIESEL. The meanings, symbols, and values of parameters are given in Table II.

[1][Online]. Available: https://github.com/NetworkCommunication/STRIVE

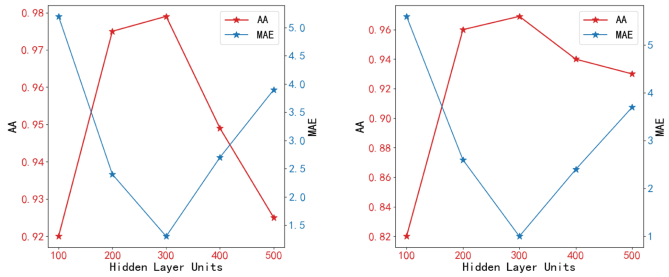


Fig. 5. Performance comparison (a) training sets (b) test sets.

B. Evaluation of Pre-Processing Models

In this section, the performance of the DT-based pre-processing model is evaluated. The following performance indicators are introduced for evaluating the simulation results.

- *Average Accuracy (AA):* AA represents the average accuracy value of each category, and in the experiment, it means the predicted even accuracy of the vehicle trajectory. Generally, when the value is higher, the result is more accurate.

$$\text{AA} = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{|\hat{y}_i - y_i|}{N}\right) \tag{38}$$

- *Mean Absolute Error (MAE):* MAE is the average absolute difference between the predicted and observed values. It can directly reflect the actual value error situation.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i| \tag{39}$$

The WGAN model employs a six-layer deep neural network [46]. The initial learning rates of the generative model and the discriminator model were set at 0.02 and 0.01 in these experiments. When it comes to adversarial learning with the WGAN model, and we select incremental parameters from 100 to 500 for training. In the experiments, the value with the highest AA and the lowest MAE is selected as the number of hidden neurons used. As shown in Fig. 5, the AA reaches its maximum value when the hidden layer contains 300 neurons, while the MAE reaches its minimum value. As a result, 300 neurons are selected for the hidden layer.

Fig. 6 shows the evaluation indices of the WGAN prediction model training, which are evaluated by the AA and the MAE. The number of hidden layer neurons in our simulation environment is 300, the AA and the MAE reach the optimal value at about 50 and 400 iterations, respectively, indicating that the training effect of this model meets the requirements.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETER OF THE ADAPTIVE MODEL

Parameter	Symbol	High QoS Requirements	Time Energy Balance	High Energy Requirements
The number of vehicles	p_{total}	30	20	15
The max computing power of the vehicle	f_i^{max}	1.5	1.5	1.5
The average computing power of HV	F_{off}^{avg}	0.4	0.5	0.6
The average computing power of SV	F_{off}^{avg}	0.5	0.6	0.7
The number of SVs	p_s	1	2	2
The weight of the load degree factor	δ	0.2	0.5	0.8
The weight of the computing power factor	η	0.8	0.5	0.2

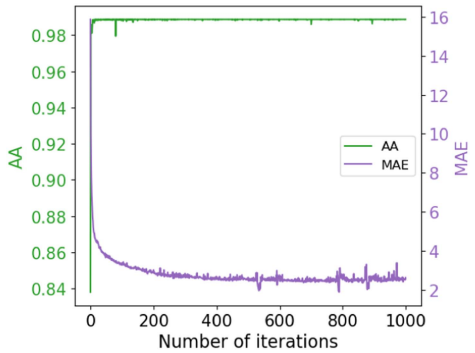


Fig. 6. Comprehensive training evaluation index of WGAN model.

C. Adaptability Evaluation Based on Dynamic Scenarios

We assess the efficacy of the DT-based adaptive model in various scenarios in this section. The experimental effects of the total cost with different weights on delay and energy consumption are shown in Fig. 7.

To cope with different scenarios in the Internet of Vehicles (IoV), we build a scenario model that is closer to the actual situation to make offloading decisions. Three distinct scenarios are employed to validate the efficacy of our adaptive model: the QoS-sensitive scenario, the energy consumption-sensitive scenario, and the balanced scenario. The principal evaluation parameters are shown in Table III. In Fig. 7(a), (b), and (c), we compare the total computation cost of three different environments under the different number of tasks, the data size of each task, and the complexity of each task. As shown, the trends of these three curves are stable under different conditions, indicating that the cost value of the adaptive model can comprehensively consider the delay and energy consumption in the actual dynamic scene.

D. Comparative Performance Evaluation

Our proposed scheme STRIVE is evaluated in comparison to the five baseline schemes that are provided below, which include None-DT Pre-processing DIESEL (NDTP-DIESEL), DT-assisted Pre-processing SAPSO (DTP-SAPSO), DT-assisted Pre-processing Greedy (DTP-GRE), DT-assisted Pre-processing GA (DTP-GA), and DT-assisted Pre-processing Local (DTP-LOC). DTP-DIESEL is defined as our scheme. We set the parameters required by pre-processing to the same values in our scenario. Furthermore, to demonstrate that our algorithm has rapid problem-solving in high dimensions and vast decision space, we compare it with the traditional Deep Q-Network (DQN) on a low-dimensional scene that can be trained.

- *NDTP-DIESEL:* This scheme adopts DIESEL in the system which is proposed in [47], but the vehicles in this scenario are not pre-processed and are randomly generated. Besides, the SA part of the algorithm does not use the adaptive annealing coefficient k. Through comparison, DT pre-processing in our scheme can reduce the size of the decision space and accelerate convergence, thereby reducing the overall cost of the system.
- *DTP-SAPSO:* Since ordinary PSO is not easy to converge in a huge decision space and difficult to add constraints, this scheme uses the SAPSO of DT pre-processing to compare with the algorithm in this paper [48]. It combines the advantages of fast convergence speed and search accuracy. Therefore, it can be proved that the convergence speed of DIESEL is fast.
- *DTP-GRE:* Each vehicle performs the offloading according to the given optimization objectives, such as first selecting the minimum fitness value [49], and then selecting the most appropriate offloading decision by selecting other vehicles or local processing. However, the strategy ignores the potential long-term consequences of offloading.
- *DTP-GA:* GA has a better ability to deal with a variety of complex capabilities and has high individual diversity and convergence accuracy in DT pre-processing scenarios. Therefore, by comparing DIESEL with GA in the same scenario, we can prove the accuracy of DIESEL's final solution.
- *DTP-LOC:* This is a type of offloading decision that only offloads tasks to itself without the assistance of other vehicles. The tasks generated by DT pre-processing for each vehicle must be offloaded locally.

Fig. 8(a) shows the total cost comparison of DTP-LOC, NDTP-DIESEL, DTP-SAPSO, DTP-GRE, DTP-GA, and DTP-DIESEL with the number of tasks, respectively. The results indicate that when the number of tasks is in the range of 10–50, with an increase in decision space, the cost of the DTP-DIESEL scheme is the lowest among the six schemes. In addition, the DTP-SAPSO scheme does not meet convergence under high-dimensional conditions with the increased number of tasks. In comparison to the alternative algorithms, DTP-DIESEL exhibits superior performance.

In Fig. 8(b), we compare the impact of six schemes on the whole system cost at different numbers of SVs and further prove the scalability of our experiment. The experimental results show that the computation cost of six schemes increases with

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

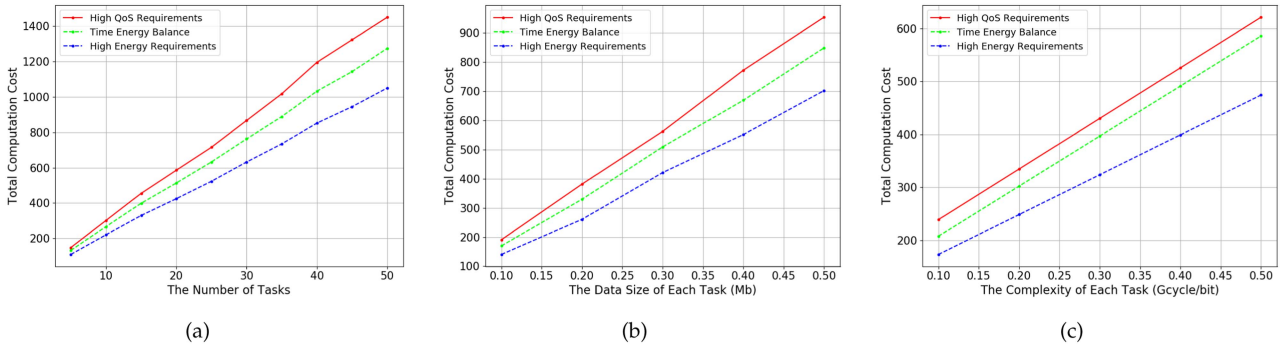


Fig. 7. Performance of the adaptive model with different task's number (a), task's size (b) and the complexity of each task (c).

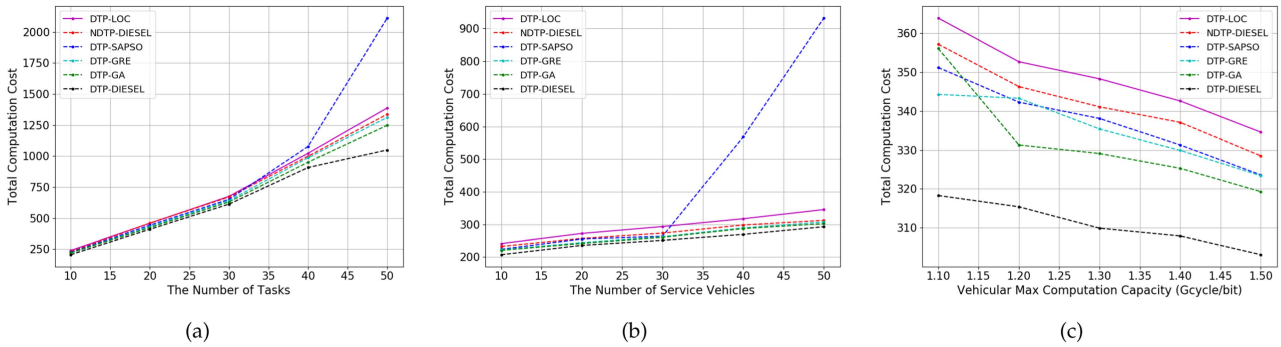


Fig. 8. Performance of different schemes with different task's number (a), service vehicle's number (b) and the vehicular max computation capacity (c).

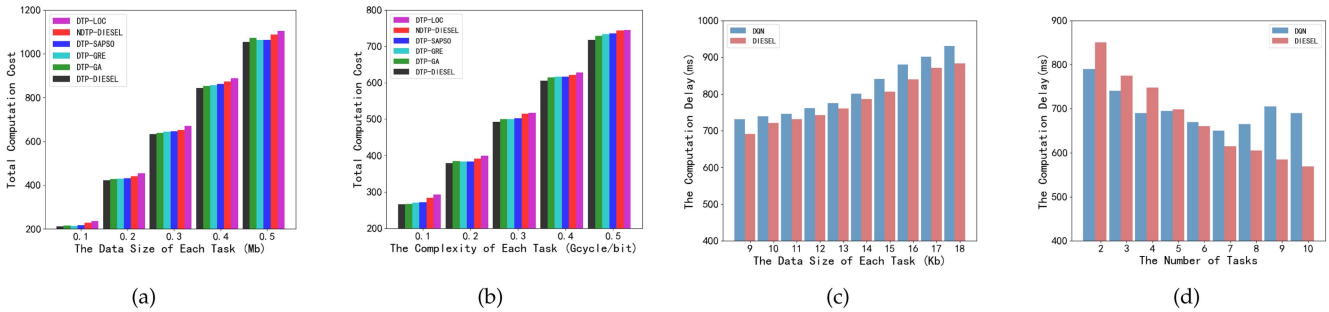


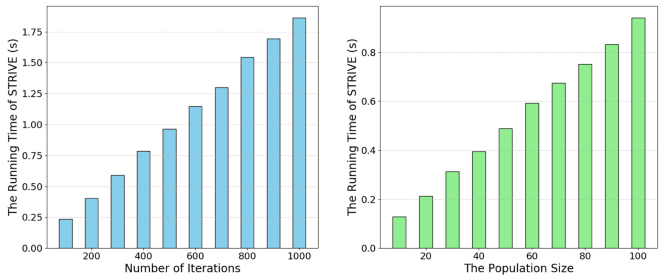
Fig. 9. (a), (b) Performance of different size of each task and complexity of each task. (c), (d) Comparing DIESEL with DQN in terms of different data size of each task and the number of tasks, respectively.

the increasing number of SVs, but DTP-DIESEL converges fast and has the lowest computation cost. The primary causes of this phenomenon are listed below. First, a DT-based pre-processing process is employed to reduce the number of offloaded vehicles needed to be judged and further reduce the decision space. Second, GA is used to make SAPSO avoid falling into local optima so that the algorithm converges more easily and obtains better solutions. Compared with other algorithms, DTP-DIESEL can obtain a better initial decision space and final offloading decision and show excellent performance in the experiment.

In Fig. 8(c), we compare the computation cost of the maximum computing power of vehicles under six scenarios. As the maximum computing power of the vehicles increases, the computing cost tends to decrease. DTP-DIESEL achieves the minimum computing cost with all degrees of computing power in vehicles. The primary cause of this phenomenon is that the maximum computing power of the vehicle has increased, resulting in more computing resources and higher computing performance. However, local offloading may not be performed in parallel by the SVs. Other schemes are inferior to DTP-DIESEL in the accuracy of the offloading decision without pre-processing. In conclusion, in terms of the total computation cost, the DTP-DIESEL scheme exhibits superior performance compared to alternative schemes.

Next, we consider the total computation cost of six different scenarios with different task sizes and task complexity. As shown in Fig. 9(a) and (b), the x-coordinate represents task size and task complexity, respectively. With the increasing size and complexity of tasks, computational and communication resources show an ascending trend. Apparently, the total cost of the

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 10. Actual running time of STRIVE (a) the number of iterators (b) the population size.

DTP-DIESEL scheme is the lowest because it continues to grow with the increasing task scale and the increasing complexity of the environment. DTP-DIESEL has a better convergence effect than other schemes. Moreover, due to the more accurate decision space for pre-processing, DTP-DIESEL has better performance than other schemes.

To verify the decision-making accuracy and quick response capability of our proposed algorithm, we conduct our comparative experiment with DRL-based algorithms in a specific scene with a sparse road segment, which selected five vehicles. As the number and size of tasks are varied to judge the superiority between algorithms, the pre-processing process is not considered due to the small size of the scene. In Fig. 9(c), we compare the total computation delay of DIESEL and DQN algorithms under different tasks. The x-coordinate is the size of the task, and the ordinate is the total computation delay, including transmission delay, computation delay, and offloading delay. It is obvious that as the task size increases, the total computation delay of DIESEL is lower than that of DQN. Because in small-scale scenarios, DQN has insufficient stability and the solutions obtained based on different training scales are unstable, DIESEL can quickly find accurate solutions in a small decision space. Therefore, the excellent performance of the DIESEL is demonstrated.

In Fig. 9(d), we compare the total computation delay of two algorithms under different numbers of tasks. The total number of tasks is 0.5 MB, and we split the task. The purpose of task splitting is to distribute the tasks to more vehicles for computation and further improve the utilization rate of IoV. In the DQN algorithm, when there are few computational tasks, the distributed offload scheme is completely offloaded to a particular vehicle. The vehicle's resources are underutilized, which increases the overall execution time. A task offloading solution is suboptimal if it consists of an excessive number of trivial tasks. The results presented by the DIESEL show that, due to the characteristics of parallel task execution, the greater the degree of task splitting, the larger the decision space, the shorter the total execution time, and the better the DIESEL's performance in high-dimensional space.

In Fig. 10, the actual running time of the algorithm is influenced by the problem complexity, which in turn is related to the number of iterations and the population size. We list the actual running time of DIESEL for various iteration counts and population sizes. We fix the population size and iteration number at 200 and 40, respectively. It is evident that the average duration of operation for STRIVE is less than one second. This is because the offloading problem has fully converged when it is iterated 200 times and the population size reaches 40, making the minimum actual running time of the problem 0.4 seconds.

E. Summary of Results

We evaluate the advantage of STRIVE by comparing it to basic algorithms, swarm intelligence algorithms, and RL algorithms. In the basic algorithms, when compared with the local offloading and greedy schemes, they only consider the short-term optimal scheme, neglecting the long-term optimal approach. In terms of swarm intelligence algorithms, we compared the DTP-DIESEL with other benchmark schemes, where the NDTP-DIESEL is a relatively novel approach but lacks a DT pre-processing and a dynamic annealing coefficient.

In order to validate the advantage of STRIVE, we compare it to the DQN algorithm for reinforcement learning in small-scale scenarios. By contrasting algorithms based on different principles, we can more convincingly demonstrate that our proposed STRIVE has a rapid problem-solving capability and a broad adaptability. Simulation results indicate that STRIVE's average cost and total delay under various conditions are superior to other task offloading schemes.

VI. Conclusion

We propose an adaptive offloading scheme in DT-assisted VEC network. The total computation cost is transformed into a combined optimization problem consisting of system delay and energy consumption by considering the dynamic nature of a VEC network environment. The proposed scheme aims to provide reliable offloading decision-making for vehicular tasks in various practical scenarios. The overall STRIVE can be divided into three parts. First, we utilize the combination of DT and WGAN to pre-process raw data from VEC network vehicles. This helps in determining the future topology of the network, thereby assisting in task offloading decisions. Second, an effective offloading scheme based on an improved swarm intelligence algorithm is applied for various dynamic scenarios, which uses the DIESEL algorithm as the method to obtain offloading decisions. Third, an adaptive model based on real-time traffic density, network load conditions, and vehicle computation power is studied to cope with the delay and energy consumption in different dynamic scenarios. We conduct an extensive experimental evaluation to prove the adaptivity, scalability, and reliability of our proposed scheme. The comparative evaluation results show that STRIVE is superior to other offloading schemes with regard to the total computation cost and the total computation delay.

References

[1] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.

[2] F. Spinelli and V. Mancuso, "Toward enabled industrial verticals in 5G: A survey on MEC-based approaches to provisioning and flexibility," *IEEE Commun. Surv. Tuts.*, vol. 23, no. 1, pp. 596–630, First Quarter 2021.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

[3] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for Internet of Things realization," *IEEE Commun. Surv. Tut.*, vol. 20, no. 4, pp. 2961–2991, Fourth Quarter 2018.

[4] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, pp. 1145–1168, Jul. 2020.

[5] M. Gupta, J. Benson, F. Patwa, and R. Sandhu, "Secure V2V and V2I communication in intelligent transportation using cloudlets," *IEEE Trans. Serv. Comput.*, vol. 15, no. 4, pp. 1912–1925, Jul./Aug. 2022.

[6] J. Thota, N. F. Abdullah, A. Doufexi, and S. Armour, "V2V for vehicular safety applications," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 6, pp. 2571–2585, Jun. 2020.

[7] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Commun. Surv. Tut.*, vol. 24, no. 4, pp. 2230–2254, Fourth Quarter 2022.

[8] K. Zhang, J. Cao, S. Maharjan, and Y. Zhang, "Digital twin empowered content caching in social-aware vehicular edge networks," *IEEE Trans. Computat. Social Syst.*, vol. 9, no. 4, pp. 239–251, Feb. 2022.

[9] W. Sun, M. Tang, L. Zhang, Z. Huo, and L. Shu, "A survey of using swarm intelligence algorithms in IoT," *Sensors*, vol. 20, Mar. 2020, Art. no. 1420.

[10] A. Shakarami, M. Ghobaei-Arani, and A. Shahidinejad, "A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective," *Comput. Netw.*, vol. 182, Dec. 2020, Art. no. 107496.

[11] A. Goldwaser and M. Thielscher, "Deep reinforcement learning for general game playing," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 1701–1708.

[12] Y. Deng, P. Wang, and L. Li, "Task offloading for mixed cloud/fog computing in vehicular network using genetic particle swarm optimization," in *Proc. 4th Int. Conf. Inf. Commun. Signal Process.*, 2021, pp. 489–494.

[13] J. Wu, Z. Huang, J. Thoma, D. Acharya, and L. V. Gool, "Wasserstein divergence for GANs," in *Proc. Eur. Conf. Comput. Vis.*, Springer International Publishing, 2018, pp. 673–688.

[14] L. Zhao, Y. Liu, A. Y. Al-Dubai, A. Y. Zomaya, G. Min, and A. Hawbani, "A novel generation-adversarial-network-based vehicle trajectory prediction method for intelligent vehicular networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 2066–2077, Feb. 2021.

[15] L. Zhao et al., "Meson: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, 2023.

[16] J. Li, B. Gu, Z. Qin, and Y. Han, "Graph tasks offloading and resource allocation in multi-access edge computing: A DRL-and-optimization-aided approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3707–3718, Nov./Dec. 2023.

[17] M. Li, J. Gao, L. Zhao, and X. Shen, "Deep reinforcement learning for collaborative edge computing in vehicular networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1122–1135, Dec. 2020.

[18] Y. Ouyang, "Task offloading algorithm of vehicle edge computing environment based on dueling-DQN," *J. Phys.: Conf. Ser.*, vol. 1873, 2021, Art. no. 012046.

[19] Y. Xia, H. Zhang, X. Zhou, and D. Yuan, "Location-aware and delay-minimizing task offloading in vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16266–16279, Dec. 2023.

[20] H. Ko, J. Kim, D. Ryoo, I. Cha, and S. Pack, "A belief-based task offloading algorithm in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 5, pp. 5467–5476, May 2023.

[21] Z. Zhang and F. Zeng, "Efficient task allocation for computation offloading in vehicular edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5595–5606, Mar. 2023.

[22] H. Guo, L.-L. Rui, and Z.-P. Gao, "V2V task offloading algorithm with LSTM-based spatiotemporal trajectory prediction model in SVCNs," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 11017–11032, Oct. 2022.

[23] X. Dai et al., "A learning-based approach for vehicle-to-vehicle computation offloading," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 7244–7258, Apr. 2023.

[24] B. Fan, Z. He, Y. Wu, J. He, Y. Chen, and L. Jiang, "Deep learning empowered traffic offloading in intelligent software defined cellular V2X networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13328–13340, Nov. 2020.

[25] J. Shi, J. Du, Y. Shen, J. Wang, J. Yuan, and Z. Han, "DRL-based V2V computation offloading for blockchain-enabled vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3882–3897, Jul. 2023.

[26] J. Shi, J. Du, J. Wang, and J. Yuan, "Deep reinforcement learning-based V2V partial computation offloading in vehicular fog computing," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2021, pp. 1–6.

[27] F. Liu, J. Chen, Q. Zhang, and B. Li, "Online MEC offloading for V2V networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6097–6109, Oct. 2023.

[28] H. Xiao, J. Zhao, J. Feng, L. Liu, Q. Pei, and W. Shi, "Joint optimization of security strength and resource allocation for computation offloading in vehicular edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8751–8765, Dec. 2023.

[29] W. Feng, S. Lin, N. Zhang, G. Wang, B. Ai, and L. Cai, "Joint C-V2X based offloading and resource allocation in multi-tier vehicular edge computing system," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 432–445, Feb. 2023.

[30] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Low-latency federated learning and blockchain for edge association in digital twin empowered 6G networks," *IEEE Trans. Ind. Inform.*, vol. 17, no. 7, pp. 5098–5107, Jul. 2021.

[31] T. Liu, L. Tang, W. Wang, Q. Chen, and X. Zeng, "Digital-twin-assisted task offloading based on edge collaboration in the digital twin edge network," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1427–1444, Jan. 2022.

[32] X. Xu et al., "Service offloading with deep Q-network for digital twinning-empowered Internet of Vehicles in edge computing," *IEEE Trans. Ind. Inform.*, vol. 18, no. 2, pp. 1414–1423, Feb. 2022.

[33] E. Zhang, L. Zhao, N. Lin, W. Zhang, A. Hawbani, and G. Min, "Cooperative task offloading in cybertwin-assisted vehicular edge computing," in *Proc. IEEE 20th Int. Conf. Embedded Ubiquitous Comput.*, 2022, pp. 66–73.

[34] B. Cao, Z. Li, X. Liu, Z. Lv, and H. He, "Mobility-aware multiobjective task offloading for vehicular edge computing in digital twin environment," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3046–3055, Oct. 2023.

[35] L. Zhao et al., "A digital twin-assisted intelligent partial offloading approach for vehicular edge computing," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3386–3400, Nov. 2023.

[36] X. Yuan, J. Chen, N. Zhang, J. Ni, F. R. Yu, and V. C. Leung, "Digital twin-driven vehicular task offloading and irs configuration in the Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24290–24304, Dec. 2022.

[37] R. P. McDonald, "Numerical methods for polynomial models in nonlinear factor analysis," *Psychometrika*, vol. 32, no. 1, pp. 77–112, 1967.

[38] E. D. L. Rosa and W. Yu, "Restricted Boltzmann machine for nonlinear system modeling," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl.*, 2015, pp. 443–446.

[39] D. Wang, D. Tan, and L. Liu, "Particle swarm optimization algorithm: An overview," *Soft Comput.*, vol. 22, pp. 387–408, Jan. 2017.

[40] M. Srinivas and L. Patnaik, "Genetic algorithms: A survey," *Computer*, vol. 27, pp. 17–26, jun 1994.

[41] N. Collins, R. Eglese, and B. Golden, "Simulated annealing an annotated bibliography," *Amer. J. Math. Manage. Sci.*, vol. 8, pp. 209–307, Feb. 1988.

[42] A. Panda and S. Pani, "A symbiotic organisms search algorithm with adaptive penalty function to solve multi-objective constrained optimization problems," *Appl. Soft Comput.*, vol. 46, pp. 344–360, Sep. 2016.

[43] A. Desai, A. Phanishayee, S. Qadeer, and S. A. Seshia, "Compositional programming and testing of dynamic distributed systems," *Proc. ACM Program. Lang.*, vol. 2, pp. 1–30, Oct. 2018.

[44] Y. Wei, M.-M. Zhao, M. Hong, M.-J. Zhao, and M. Lei, "Learned conjugate gradient descent network for massive MIMO detection," *IEEE Trans. Signal Process.*, vol. 68, pp. 6336–6349, 2020.

[45] F. Pontes, G. Amorim, P. Balestrassi, A. Paiva, and J. Ferreira, "Design of experiments and focused grid search for neural network parameter optimization," *Neurocomputing*, vol. 186, pp. 22–34, Apr. 2016.

[46] L. Zhao et al., "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3848–3858, Sep. 2020.

[47] J. Bi, H. Yuan, S. Duanmu, M. Zhou, and A. Abusorrah, "Energy-optimized partial computation offloading in mobile-edge computing with genetic simulated-annealing-based particle swarm optimization," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3774–3785, Mar. 2021.

[48] H. Yuan, J. Bi, W. Tan, and B. H. Li, "Temporal task scheduling with constrained service delay for profit maximization in hybrid clouds," *IEEE Trans. Automat. Sci. Eng.*, vol. 14, no. 1, pp. 337–348, Jan. 2017.

[49] F. Wei, S. Chen, and W. Zou, "A greedy algorithm for task offloading in mobile edge computing system," *China Commun.*, vol. 15, pp. 149–157, Nov. 2018.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.



Liang Zhao (Member, IEEE) received the PhD degree from the School of Computing, Edinburgh Napier University, in 2011. He is a professor with Shenyang Aerospace University, China. Before joining Shenyang Aerospace University, he worked as associate senior researcher with Hitachi (China) Research and Development Corporation from 2012 to 2014. He is also a JSPS Invitational fellow (2023–2024). He was listed as Top 2% of scientists in the world by Standford University (2022 and 2023). His research interests include ITS, VANET, WMN, and SDN. He has published more than 170 articles. He served as the chair of several international conferences and workshops, including 2022 IEEE BigDataSE (Steering Co-Chair), 2021 IEEE TrustCom (Program Co-Chair), 2019 IEEE IUCC (Program Co-Chair), and 2018-2022 NGDN workshop (founder). He is currently serving on the editorial boards of many international journals. He was the recipient of the Best/Outstanding Paper Awards at 2015 IEEE IUCC, 2020 IEEE ISPA, 2022 IEEE EUC, and 2013 ACM MoMM.



Tianyu Li is currently working toward the MS degree with Shenyang Aerospace University. His research interests include mobile edge computing, computation offloading and digital-twins.



Enchao Zhang is currently working toward the MS degree with Shenyang Aerospace University. His research interests include mobile edge computing, computation offloading, digital-twins, vehicular network, and resource allocation.



Yun Lin (Senior Member, IEEE) received the BS degree from Dalian Maritime University, Dalian, China, in 2003, the MS degree from the Harbin Institute of Technology, Harbin, China, in 2005, and the PhD degree from Harbin Engineering University, Harbin, in 2010. He was a research scholar with Wright State University, USA, from 2014 to 2015. Now, he is currently a full professor with the College of Information and Communication Engineering, Harbin Engineering University, China. His current research interests include machine learning and data analytics over wireless networks, signal processing and analysis, cognitive radio and software defined radio, artificial intelligence, and pattern recognition. He had published more than 200 international peer-reviewed journal/conference papers, such as the *IEEE Transactions on Signal Processing*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Communications*, *IEEE Internet of Things Journal*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Cognitive Communications and Networking*, *IEEE Transactions on Reliability*, INFOCOM, GLOBECOM, ICC, VTC, ICNC. He is serving as an editor for the *IEEE Transactions on Reliability*, *Digital Communications and Networks*, *KSII Transactions on Internet and Information Systems*, and *International Journal of Performability Engineering*. He serves as GC2022 co-chair of Mobile and Wireless Networking Symposium, General Vice Chair of VTC-2021 Fall, general chair of ADHIP 2020, TPC chair of MOBIMEDIA 2020, ICEICT 2019 and ADHIP 2017, and TPC member of GLOBECOM, ICC, ICNC, WCSP, and VTC. He is a recipient of IEEE Outstanding service award of Trustcom 2021, IEEE Outstanding Track Chair Award of MASS 2021.



Shaohua Wan (Senior Member, IEEE) received the PhD degree from the School of Computer, Wuhan University, in 2010. He is currently a professor with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China. From 2016 to 2017, he was a visiting professor with the Department of Electrical and Computer Engineering, Technical University of Munich, Germany. His main research interests include deep learning for Internet of Things. He is an author of more than 150 peer-reviewed research papers and books, including more than 40 IEEE/ACM Transactions papers, such as *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Intelligent Transportation Systems*, *ACM Transactions on Internet Technology*, *IEEE Transactions on Network Science and Engineering*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Computational Social Systems*, *ACM Transactions on Multimedia Computing, Communications, and Applications*, *IEEE Transactions on Emerging Topics in Computational Intelligence*, PR, etc., and many top conference papers in the fields of edge intelligence.



Ammar Hawbani received the BS degree in computer software and theory and the MS and a PhD degrees from the University of Science and Technology of China (USTC), in 2009, 2012, and 2016, respectively. is a full professor with the School of Computer Science, Shenyang Aerospace University. Following his PhD completion, he served as a postdoctoral researcher with the School of Computer Science and Technology, USTC from 2016 to 2019. Later, he worked as an associate researcher with the School of Computer Science and Technology, USTC from 2019 to 2023. Currently, he holds the position of full professor with the School of Computer Science in Shenyang Aerospace University. His research interests span IoT, WSNs, WBANs, WMNs, VANETs, and SDN.



Mohsen Guizani (Fellow, IEEE) received the BS (with distinction), MS and PhD degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA, in 1985, 1987 and 1990, respectively. He is currently a professor of machine learning with the Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, UAE. Previously, he worked in different institutions in USA. His research interests include applied machine learning and artificial intelligence, smart city, Internet of Things (IoT), intelligent autonomous systems, and cybersecurity. He was listed as a Clarivate Analytics Highly Cited researcher in computer science in 2019, 2020, 2021 and 2022. He has won several research awards including the "2015 IEEE Communications Society Best Survey Paper Award", the Best ComSoc Journal Paper Award in 2021 as well 5 Best Paper Awards from ICC and Globecom Conferences. He is the author of 11 books, more than 1000 publications and several US patents. He is also the recipient of the 2017 IEEE Communications Society Wireless Technical Committee (WTC) Recognition Award, the 2018 AdHoc Technical Committee Recognition Award, and the 2019 IEEE Communications and Information Security Technical Recognition (CISTC) Award. He served as the editor-in-chief of IEEE Network and is currently serving on the editorial boards of many IEEE Transactions and Magazines. He was the chair of the IEEE Communications Society Wireless Technical Committee and the chair of the TAOS Technical Committee. He served as the IEEE Computer Society distinguished speaker and is currently the IEEE ComSoc distinguished lecturer.

Authorized licensed use limited to: UNIVERSITY OF ELECTRO COMMUNICATIONS. Downloaded on April 07,2025 at 11:23:02 UTC from IEEE Xplore. Restrictions apply.

View publication stats